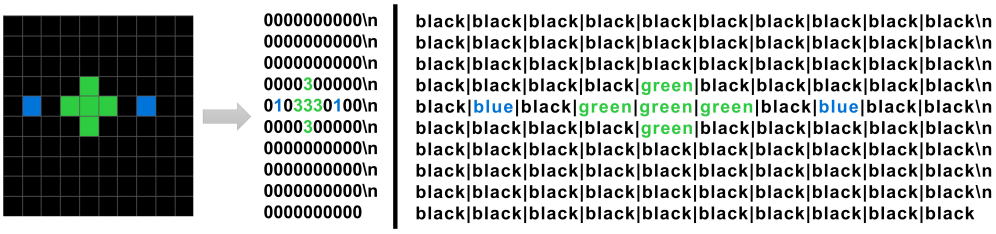

Figure 3: **Visualization of direct-grid encodings.** Left: each pixel is represented with a number corresponding to the pixel color, with no delimiters. Right: each pixel is represented with the color descriptor, separated by the delimiter "|". The text string has been formatted for easier reading.

2 A first attempt at solving ARC tasks with an LLM

The task of solving the ARC with an LLM necessitates the encoding of two-dimensional (2D) input-output images using a textual representation[2]. A text-encoded ARC task is incorporated into an LLM prompt, which then generates the solution. This section proposes a straightforward pipeline and evaluates it before mining the results to understand failure modes.

2.1 Textual encoding

A 2D grid with colored pixels can be directly encoded into text by representing each pixel's color either numerically (using values from 0 to 9, each representing one of the ten colors) or with color descriptors (e.g., "blue", "green", "black"). A delimiter delineates between adjacent pixels and "newline" characters were used to separate the rows in an image. We assessed the impact of different delimiters ("**,**", "|", or no delimiter) on LLM performance. Figure 3 provides two visual examples of this *direct-grid encoding*.

2.2 Prompting and strategy

After encoding ARC images into text, we incorporate the latter into prompts that instruct the LLM to solve the task at hand. We explored two single-stage strategies for prompting the LLM.

Few-shot learning: By design, an ARC task is a few-shot learning task, providing a handful of training examples for generating a solution for a test example. Therefore, adopting the few-shot learning strategy is the most straightforward and intuitive initial approach for leveraging the inherent structure of ARC tasks. The prompt created using this strategy has two main sections: "instructions" and "task". The "instructions" section outlines the nature of an ARC task and the expected behavior of the LLM; it is the same across all tasks. The "task" section provides information about the single ARC task of interest, including its few-shot examples. This approach, in line with the classic few-shot learning concept, encourages the LLM to leverage the provided examples to solve the task. An example of this prompting strategy can be found in Figure 4.

In-context few-shot learning with chain-of-thought: Building on the few-shot learning strategy and drawing inspiration from chain-of-thought (CoT) prompting introduced in (Wei et al., 2022), we investigate a natural combination thereof. This approach enriches the learning context for the LLM by augmenting the original prompt with an "examples" section, which includes two simple ARC-like tasks—different from the actual task of interest—and their step-by-step solutions. This strategy not only leverages the inherent task examples but also provides a stable learning base of *in-context examples* for the LLM, encouraging a CoT response. As such, it assesses the LLM's capacity to generalize and apply knowledge acquired from a limited set of contextual examples to solve a similar task. An example prompt using this approach can be found in Figure 4.

[2]At the time of our research, GPT-4's vision API, which enables the model to accept image inputs, was not yet available. Consequently, this paper does not focus on the multimodal capabilities of GPT-4. However, we provide glimpses into these capabilities and their potential in Section 6 and Appendix G.

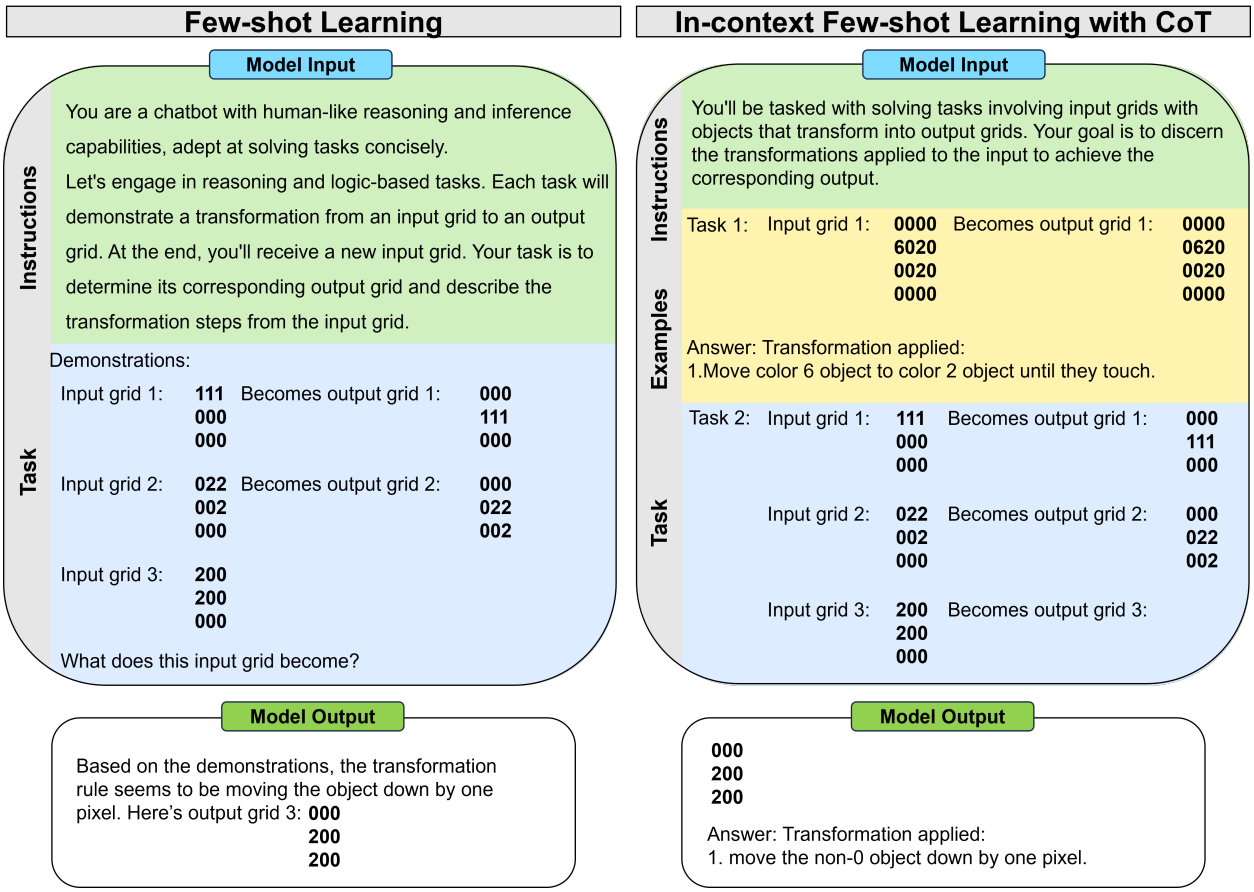

Figure 4: **Example prompts.** Left: Few-shot Learning. Right: In-context Few-shot Learning with CoT. The prompt texts have been formatted for easier reading (for example, the "Task" string on the left is provided to the LLM as "Demonstrations:\nInput grid 1: 111\n000\n000...").

Table 1: **Direct-grid variants, performance comparison.** Each row corresponds to a variant of a direct-grid encoding. Each column corresponds to a combination of a prompting method with either GPT-3.5 or GPT-4. GPT solutions were obtained through OpenAI's API with temperature set to 0. The values correspond to the number of tasks, out of 50, solved by each method; higher is better and top-performers are bolded.

Direct-grid encoding		Few-shot		In-context Few-shot w/ CoT	
Pixel	Delimeter	GPT-3.5	GPT-4	GPT-3.5	GPT-4
Number	n/a	3	5	2	9
Number	|	4	11	5	12
Word	,	3	12	5	**13**
Word	|	4	8	2	**13**

2.3 Results

Given that the most advanced ARC solvers achieve only a 30% accuracy on the hidden test set of the ARC, we strategically selected a subset of 50 ARC tasks. These tasks were among the "easiest", allowing our resources to be more efficiently allocated for in-depth experimentation. Our goal was to upper-bound the LLM's performance on the ARC as a whole; note that we do discuss results on all 400 ARC training tasks in the following paragraph and Section 4.2, but we restrict much of the analysis in this paper to the selected 50

tasks. We define "easy" tasks as those that have been previously addressed using the symbolic search-based method, ARGA (Xu et al., 2023). This designation stems from the fact that ARGA's implementation confines its solution space to 15 functions, in contrast to state-of-the-art models which can have a more expansive function set, with the Kaggle first-place solution, for example, encompassing 42 functions. Thus, studying this subset provides a clearer understanding of how a purely search-based solver, with a restricted function set, tackles ARC tasks.

The top-performing pixel representation and delimiter combination (Word + "|") solves only 13 out of the 50 tasks, as shown in Table 1. Using the top-performing representation, we assessed the complete ARC training set and GPT-4 managed to solve 81 out of the 400 tasks. Furthermore, on the Mini-ARC dataset—a simplified version of ARC detailed in Section 5—this approach solved 35 out of the 149 tasks. These accuracy rates are consistent with the performance observed on the subset of 50 tasks. In the following section, we will delve into the reasons why the LLM struggled with these tasks given that they are easy for a non-LLM method.

2.4 Analysis

We started our analysis by extracting key attributes such as pixel and color counts from each ARC task. We then applied logistic regression to explore potential relationships between these features and the performance of the LLM.

An intriguing finding from our analysis is that the number of colored pixels in a *test* image is associated with a notable negative coefficient, indicating a potential inverse relationship with the LLM's ability to solve tasks. Since a set of adjacent colored pixels often corresponds to an object in the ARC, this finding suggests that tasks with fewer objects are *more likely* to be solved by the LLM. Conversely, we find a positive coefficient associated with the average number of colored pixels in *training* images, implying a possible positive correlation with task solvability. This could suggest that more colored pixels in training images provide more learning material for the LLM, potentially improving performance. The full set of features studied can be found in Appendix E.

A closer examination of the tasks that GPT-4 solved correctly using the direct-grid approach reveals some interesting patterns in the reasoning provided by the model. Out of the 13 tasks that were correctly solved, only three tasks were accompanied by the correct reasoning steps. Surprisingly, for some tasks, GPT did not provide any reasoning at all, despite the presence of reasoning examples within the In-context Few-shot Learning with CoT prompts. This inconsistency in the application of reasoning illustrates a possible gap in GPT's understanding and application of the reasoning process, which further complicates the task of solving ARC problems. An example of a task where the reasoning provided by the model was incorrect despite achieving the correct output is illustrated in Figure 5. Further examples can be found in Appendix D.

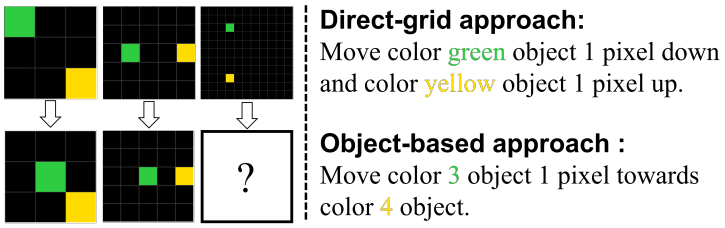

Figure 5: **Reasoning provided by GPT-4 for an example task.** Both approaches produced the correct output grid. The direct-grid approach produced the wrong reasoning while the object-based approach produced the correct one.